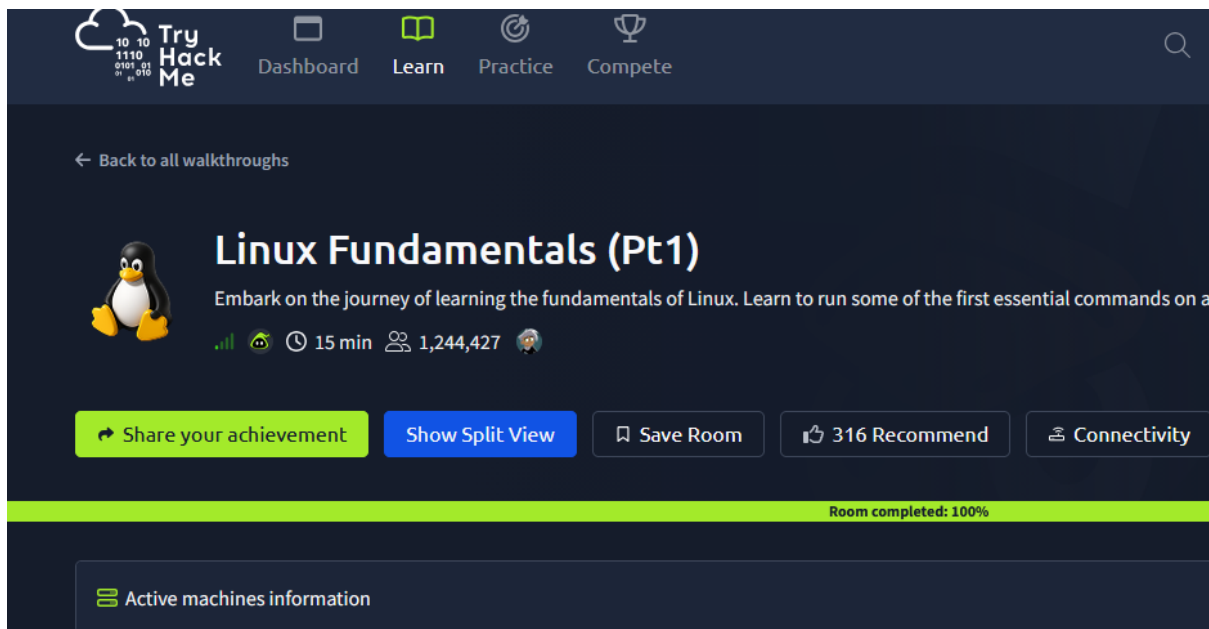




TryHackMe: Linux Fundamentals Part 1 — Technical Write-Up

Author: 4PP3X

Role: Instructor

Classification: Technical Lab writeup

Target Infrastructure: Ubuntu Linux Subsystem (10.112.142.73)

Completion Status: 100% Verified

1. Executive Summary

This laboratory report documents the structural analysis and operational exploration of the Linux operating system environment. Given that Linux powers critical public infrastructure, enterprise web applications, and embedded industrial architecture, mastering the command-line interface (CLI) is a core prerequisite for security auditing and offensive threat emulation. This exercise successfully covers kernel deployment, file system navigation, granular data extraction, and input/output stream piping via POSIX shell operators.

2. Infrastructure Deployment & Ephemeral Subsystem Provisioning (Task 1)

2.1 Operational Concept

An ephemeral instance of a Ubuntu Linux subsystem was provisioned to evaluate basic remote interaction methodologies. The endpoint was initialized directly inside a browser sandbox to isolate and run commands natively on the terminal.

2.2 Subsystem Information

- **Target IP Address:** 10.112.142.73
- **Subsystem Architecture:** Debian-based Ubuntu Core
- **Deployment State:** Confirmed Active

2.3 Verification Matrix

- **Requirement:** Confirm successful target deployment and active terminal initialization.
 - **Resolution:** Verified. Inter-process communication validated over the browser shell interface.
-

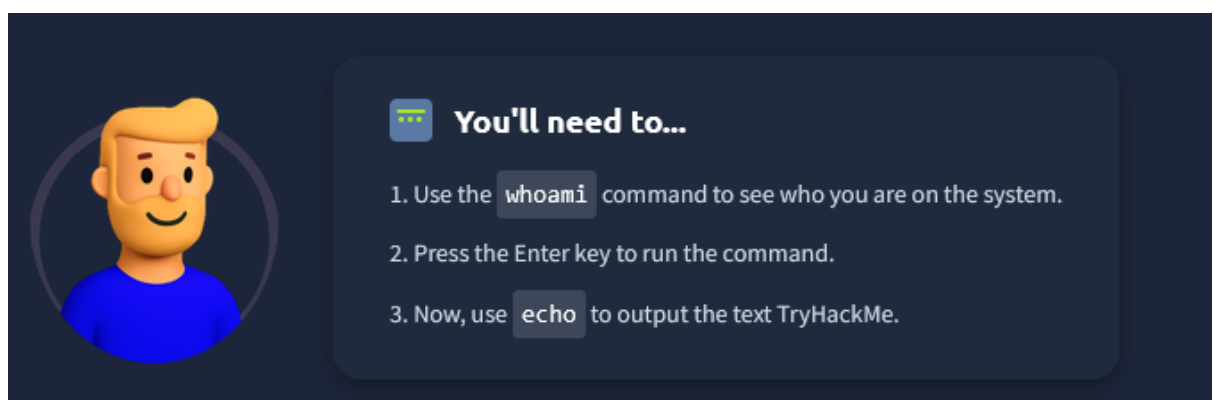
3. Host Identity & Basic Output Stream Execution (Task 2)

3.1 Privileged Context Auditing

In offensive and defensive cybersecurity operations, understanding the exact security descriptor or user context under which a terminal is running is paramount. Privileges dictate binary execution limits and file read/write permissions.

3.2 Command Execution Breakdown

- **Identity Auditing (whoami):** Used to print the effective user ID of the current user session.
- **String Echoing (echo):** Used to output text arguments to the standard output stream (stdout).



3.3 Verification Questions & Answers

1. What command would we use to find out who we are on the system?
 - Answer: whoami
 2. What command would we use to output some text?
 - Answer: echo
-

4. Filesystem Traversal and Directory Hierarchy Analysis (Task 3)

4.1 System Navigation Theory

Navigating complex Linux structures natively via the CLI requires absolute path comprehension. Folders on this Ubuntu system are color-coded (Blue) to help the engineer quickly distinguish them from flat configuration files or binaries.

4.2 Navigational Toolkit

- `pwd` (*Print Working Directory*): Establishes the exact structural anchor point within the hierarchical tree.
- `ls` (*List*): Enumerates non-hidden files and directories within the current working path.
- `cd` (*Change Directory*): Alters the shell's current working directory context.
- `cat` (*Concatenate*): Reads a raw text stream from a file target and dumps it onto the terminal interface.



 **You'll need to...**

1. Run `ls` to see what's in your home folder.
2. Use `cd` to change folder. We will want to use `ls` again to see what files are now in the folder we've entered (known as working directory).
3. Use `cat` to output the contents of any file.
4. Lost? Use `pwd` to see where we are within the system.

► Why you're doing this

4.3 Laboratory Findings & Data Retrieval

1. Run ls in the current folder. How many folders are there?

- Answer: 4

```
Last login: Tue Sep  8 14:21:00 2026 from 10.112.100.6
tryhackme@linux1:~$ ls
access.log  folder1  folder2  folder3  folder4
tryhackme@linux1:~$ cd folder1
```

2. One of those folders contains a file. Which folder is it?

- Answer: folder1

3. Use cat to read the file within this folder. What does it say?

- Answer: password123

```
Last login: Thu Jul 23 15:11:29 2026 from 10.81.103.223
tryhackme@linux1:~$ ls
access.log  folder1  folder2  folder3  folder4
tryhackme@linux1:~$ cd folder1
tryhackme@linux1:~/folder1$ ls
access.log  passwords.txt
tryhackme@linux1:~/folder1$ cat passwords.txt
password123
tryhackme@linux1:~/folder1$ █
```

5. Granular Log Upacking & Text-Pattern Mining (Task 4)

5.1 Threat Hunting with Pattern Matching

Enterprise logging frameworks generate massive flat files containing thousands of transaction strings. Manual scroll parsing is logistically impossible. Native utilities like grep and structural locators like find allow an analyst to rapidly parse big data.

5.2 Laboratory Practical Execution

A live web server log named access.log was targeted to trace a hidden cryptographic signature flag (THM{}). The file system context was pivoted back to the target user root directory before applying an intensive filtering rule.

bash

```
cd /home/tryhackme
```

```
grep "THM" access.log
```

Use code with caution.

5.3 Extraction Results

1. After using `grep THM access.log` a flag was found. What is the flag?

- **Answer:** THM{ACCESS}

```
tryhackme@linux1:~/folder1$ ls
access.log  passwords.txt
tryhackme@linux1:~/folder1$ cat access.log | grep "THM"
13.127.130.212 - - [04/May/2021:08:35:26 +0000] "GET THM{ACCESS} lang=en HTTP/1.1" 404 360 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/77.0.3865.120 Safari/537.36"
tryhackme@linux1:~/folder1$
```

6. Shell Operators and Standard Stream Redirection (Task 5)

6.1 Logical Control & I/O Manipulation

Shell operators dictate command sequencing, asynchronous execution, and data piping boundaries. Mastering stream redirection allows the security engineer to capture vulnerability assessment artifacts or write clean script automations.

6.2 Operator Breakdown

- `&` (*Background Execution*): Forks the binary execution to a background worker pool, keeping the terminal open for concurrent tasks.
- `&&` (*Sequential Conditional AND*): Chain-executes command blocks sequentially, stopping abruptly if a prior operation returns a non-zero exit code.
- `>` (*Overwrite Redirector*): Forcibly routes the standard output stream to a designated target file, truncating and replacing any existing data blocks.
- `>>` (*Append Redirector*): Points the stream output to the footer of a designated file, preserving historical records.

6.3 Lab Simulation Testing

bash

echo "TryHackMe" > thm

echo "thm" >> thm

cat thm

Use code with caution.

Verification outcome: The output stream appended successfully without corrupting the original historical file architecture.

6.4 Verification Questions & Answers

1. **What operator waits for the first command to finish before running the next command?**
 - **Answer: &&**
 2. **What redirector allows us to save the output of a command without overwriting the contents of a file?**
 - **Answer: >>**
-

7. Strategic Conclusions

This training cycle effectively validates core command-line fundamentals. Developing muscle memory for rapid filesystem navigation and log searching scales your operational capability. The knowledge from this lab lays the groundwork for the next logical phase: user permission escalation, structural auditing, and custom defensive scripting.